



TAREA: Buenas Practicas, POO y Principios SOLID en Python

Manejo de Datos

Facultad de Ciencias

Equipo: Los Bonitos

Integrantes:

De la Peña Driottz María Alejandra

Flores Sánchez Itzel Jessica

León Luna Alan Gaell

Medina Cortés Fernanda

Profesora: Jessica Santizo Galicia

Ayudante: Brenda Lizeth Ramírez Arista

1.1 Parte A - PEP8 (PEP 8 – Style Guide for Python Code)

1. Sangría: Se deben dejar 4 espacios por cada nivel de indentación dentro de un bloque de código. También se utiliza en sentencias que ocupan más de una línea, como en las funciones, para mantener la legibilidad y estructura del código.

```
#ANTES:
def par(num):
    if num % 2 == 0:
        return True
    return False

#DESPUÉS:
def par(num):
    if num % 2 == 0:
        return True
    return False
```

2. Longitud máxima de línea: Una línea puede tener como máximo 79 caracteres. Los comentarios y cadenas de documentación deben limitarse a 72 caracteres. Esto permite tener más de una ventana abierta al mismo tiempo.

```
#ANTES:
def funcion(primer_parametro, segundo_parametro='opcional', tercer_parametro=None):
    return True

#DESPUÉS:
if peso > 93 and dias_hasta_verano < 100 \
    and destino_vacaciones == 'Playa':
    hacer_ejercicio()
```

3. Líneas en blanco: Las definiciones de funciones y clases de nivel superior se separan con dos líneas en blanco. Las definiciones de métodos se separan con una línea en blanco. Para separar funciones relacionadas se puede usar una línea en blanco adicional, o bien podemos omitirlas si varias líneas de código están muy relacionadas entre sí.

```
#ANTES:
class Calculadora:
    def __init__(self, valor_inicial=0):
        self.resultado = valor_inicial
    def sumar(self, numero):
        self.resultado += numero
        return self.resultado

#DESPUÉS:
class Calculadora:

    def __init__(self, valor_inicial=0):

        self.resultado = valor_inicial

    def sumar(self, numero):
        self.resultado += numero
        return self.resultado
```

4. Codificación del archivo fuente: El código de Python siempre debe usar UTF-8 sin declaraciones de codificación. Las codificaciones que sean distintas a este, solo se permiten en la biblioteca estándar para pruebas. Limitar el uso de caracteres no ASCII a nombres propios, lugares o datos limpios. A su vez, todos los identificadores deben estar en ASCII y preferentemente en inglés.

```
#ANTES:
def calcular_puntuación_público(puntuación_base)
    lugar = "España"
    mensaje_error = "Error: ẑalgo"
    return puntuación_base * 1.5

#DESPUÉS:
def calculate_public_score(base_score):
    location = "España"
    error_message = "Error: Invalid input"
    return base_score * 1.5
```

5. Importaciones: Las importaciones se colocan al inicio del código y deben ir en líneas separadas. Se deben agrupar en el siguiente orden: primero la biblioteca estándar, después las librerías de terceros y, por último, las importaciones locales. Se recomienda usar importaciones absolutas; las relativas solo deben usarse si son explícitas y, preferiblemente, simples y cortas.

```
#ANTES:
import json, sys

#DESPUÉS:
import json
import sys
```

6. Nombres de Dunder a nivel de módulo: Los nombres a nivel de módulo que tienen guiones bajos al inicio y al final se deben colocar después de la cadena de documentación del módulo y antes de las importaciones.

```
#ANTES:
author = "Nombre"
version = "1.0"
all = ["funcion", "Clase"]

#DESPUÉS:
__author__ = "Nombre"
__version__ = "1.0"
__all__ = ["funcion", "Clase"]
```

7. Citas de cadena: Usar comillas simples o dobles para definir una cadena es exactamente igual. Si la cadena contiene un tipo de comillas, se pueden usar las otras para evitar el uso de barras invertidas. En el caso de comillas triples, se deben usar comillas dobles, siguiendo la convención para *docstrings* establecida en la PEP 257.

#ANTES:

```
mensaje = "El usuario dijo: Hola"  
texto = "It's fine"
```

#DESPUÉS:

```
mensaje = 'El usuario dijo: "Hola"'  
texto = "It's fine"
```

8. Espacios en blanco: No se deben dejar espacios en blanco innecesarios, ya sea dentro de paréntesis, corchetes o llaves, ni antes de una coma, punto o punto y coma.

#ANTES:

```
funcion( x , y )  
lista = [ 1, 2, 3 ]  
diccionario = { 'clave': 10 }
```

#DESPUÉS:

```
funcion(x, y)  
lista = [1, 2, 3]  
diccionario = {'clave': 10}
```

9. Comentarios: No deben contradecir la línea original del código, por lo que se deben ir actualizando según este vaya cambiando. Deben ser oraciones completas y comenzar con mayúscula; si es una oración, debe terminar con un punto. Todos los comentarios tienen que ser comprensibles.

#ANTES:

```
x = x + 1 # incrementa x por dos
```

#DESPUÉS:

```
x += 1 # Incrementa la variable x una unidad.
```

10. Comas al final de la oración: Las comas al final suelen ser opcionales, salvo cuando creas una tupla de un solo elemento, aquí es obligatoria y se recomienda usar paréntesis para que sea claro.

#ANTES:

```
FILES = 'setup.cfg',
```

#DESPUÉS:

```
FILES = ('setup.cfg',)
```

¿Qué es el conjunto de principios SOLID?

Es un conjunto de 5 principios que son utilizados para la OOP (Programación Orientada a Objetos), que ayudan a crear un código más limpio y eficiente. Estos principios se pueden utilizar para proyectos de software, se utilizan en cualquier parte del código como por ejemplo al definir una clase o método.

1. S Single Responsibility Principle (Principio de responsabilidad única)

El problema que busca resolver, es evitar que una sola clase haga distintas cosas a la vez de forma que si le hacemos un cambio a la clase, no afecte y no modifique a otras partes del código o del sistema.

Este principio nos dice que una clase debería de tener una sola responsabilidad específica.

EJEMPLO:

```
# CODIGO MALO
class GestorBoletos:
    def vender(self, comprador, precio):
        print(f"Boleto generado para {comprador}")
        print(f"Correo enviado a {comprador}")
        print(f"Venta registrada: {precio}")

# CODIGO BUENO
class GeneradorBoletos:
    def generar(self, comprador):
        print(f"Boleto generado para {comprador}")

class Notificador:
    def enviar(self, comprador):
        print(f"Correo enviado a {comprador}")

class Registrador:
    def registrar(self, precio):
        print(f"Venta registrada: {precio}")
```

Cita: Baruri, K. C. (2023, 22 de abril). *SOLID principles of OOP*. Medium. <https://medium.com/@kc.baruri/solid-principles-of-oop-d844ab130276>

2. O Open/Closed Principle (Principio de abierto/cerrado)

El problema que busca resolver, es evitar que cambiemos el código antiguo cuando agreguemos algo nuevo, de forma que si agregamos cambios al código no afecte lo demás que ya funcionaba.

Este principio nos dice que las clases de un software o código deben de estar diseñadas para estar abiertas para poder extenderse, pero cerradas para modificarse.

EJEMPLO:

```
# CODIGO MALO
class Comision:
    def calcular(self, tipo, ventas):
        if tipo == "planta":
            return ventas * 0.05
        elif tipo == "freelance":
            return ventas * 0.12
        # cada tipo nuevo = editar esta funcion

# CODIGO BUENO
class ComisionPlanta:
    def calcular(self, ventas):
        return ventas * 0.05

class ComisionFreelance:
    def calcular(self, ventas):
        return ventas * 0.12

# tipo nuevo= clase nueva, no tocas las anteriores

class ComisionSocio:
    def calcular(self, ventas):
        return ventas * 0.08
```

Cita: Programador Novato. (2023, 2 de octubre). *Los 5 principios SOLID con ejemplos | Explicación completa en 10 minutos* [Video]. YouTube. <https://www.youtube.com/watch?v=wAW7ldVGfLo>

3. L Liskov Substitution Principle (Principio de sustitución de Liskov)

El problema que busca resolver, es que una clase hija herede la estructura de la clase padre y que cumpla con sus características de comportamiento. Cuando la clase hija hereda la estructura pero no cumple con las características de la clase padre el código falla.

Este principio nos dice que una clase hija debe poder sustituir a un objeto de la clase padre sin generar errores, ni modificar el comportamiento de la clase padre.

EJEMPLO:

```
# CODIGO MALO
class Cerradura:
    def abrir(self):
        print("Abierta")

class CerraduraEnMantenimiento(Cerradura):
    def abrir(self):
        raise Exception("No se puede abrir")

# rompe con lo que el padre prometia

# CODIGO BUENO
class Cerradura:
    def abrir(self):
        print("Abierta")
        return True

class CerraduraEnMantenimiento(Cerradura):
    def abrir(self):
        print("No se pudo abrir")
        return False
```

Cita: Akbary, K. (s.f.). *Principio de sustitución de Liskov*. Keyvan Akbary.
<https://keyvanakbary.com/principio-de-sustitucion-de-liskov/>

4.1 Interface Segregation Principle (Principio de Segregación de Interfaces)

Este principio busca evitar que creamos clases obligadas a incluir funciones que realmente no necesitan solo porque una interfaz general se las impuso. En palabras sencillas, nos dice que es mucho mejor diseñar varias interfaces pequeñas y específicas en lugar de una sola interfaz gigante con demasiados métodos; así evitamos que el código quede sobrecargado con métodos vacíos o que lancen errores por falta de permisos.

EJEMPLO:

(malo)

```
1 from abc import ABC, abstractmethod
2
3
4 class OperacionesSeguro(ABC):
5     @abstractmethod
6     def cotizar(self):
7         pass
8
9     @abstractmethod
10    def autorizar_pago_siniestro(self):
11        pass
12
13
14 class AgenteVentas(OperacionesSeguro):
15     def cotizar(self):
16         return "Cotización generada"
17
18     def autorizar_pago_siniestro(self):
19         # Viola ISP: se le obliga a implementar algo ajeno a su rol
20         raise PermissionError("Sin permisos para autorizar pagos")
```

(bueno)

```

1 from abc import ABC, abstractmethod
2
3
4 class Cotizable(ABC):
5     @abstractmethod
6     def cotizar(self):
7         pass
8
9
10 class Autorizable(ABC):
11     @abstractmethod
12     def autorizar_pago_siniestro(self):
13         pass
14
15
16 # El agente solo implementa lo que le corresponde
17 class AgenteVentas(Cotizable):
18     def cotizar(self):
19         return "Cotización generada"

```

Cita: IONOS. (2025, 14 de enero). *Los principios SOLID de un vistazo: explicación con ejemplos.*
 IONOS Digital Guide.

<https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/solid-principles/>

5. D Dependency Inversion Principle (Principio de Inversión de Dependencias)

Este principio resuelve el problema del alto acoplamiento, el cual ocurre cuando la lógica principal de nuestro programa depende directamente de herramientas secundarias específicas (como una librería o servicio externo en particular) y cualquier cambio técnico nos obliga a reescribir todo el sistema. Básicamente propone que las clases importantes no dependan de clases concretas sino de contratos abstractos (interfaces), permitiendo cambiar o actualizar los detalles técnicos sin alterar la lógica central del negocio.

EJEMPLO:

(malo)

```

1 class BaseDeDatosMySQL:
2     def guardar(self, dato: str):
3         print("Guardando en MySQL")
4
5 class GestorUsuarios:
6     def __init__(self):
7         # Dependencia rígida de una implementación concreta
8         self.db = BaseDeDatosMySQL()
9
10    def registrar(self, usuario: str):
11        self.db.guardar(usuario)

```

(bueno)

```

1 from abc import ABC, abstractmethod
2
3 class Repositorio(ABC):
4     @abstractmethod
5     def guardar(self, dato: str): pass
6
7 class BaseDeDatosMySQL(Repositorio):
8     def guardar(self, dato: str):
9         print("Guardando en MySQL")
10
11 class GestorUsuarios:
12     # Inyección de dependencia a través de la abstracción
13     def __init__(self, repo: Repositorio):
14         self.repo = repo
15
16     def registrar(self, usuario: str):
17         self.repo.guardar(usuario)

```

Cita: Deiner, H. (2018, 27 de junio). *SOLID principles explained by examples of bad and good code* [Repositorio]. GitHub. <https://github.com/hdeiner/SOLID>

2. Aseguradora

¿Qué principio aplica cada uno de los puntos anteriores?

1.- Validación robusta:

En este punto se implementa la validación de datos introducidos por el usuario para evitar que el programa continúe con información incorrecta. Por ello utilizamos excepciones personalizadas como `EdadInvalididad_error`, etc. Como ejemplo la validación de edad se realiza mediante el método `pedir_edad()` de nuestra clase entrada de datos, donde verificamos que la edad se encuentre entre los 18 y 99 años como especifica el programa. Y no solo eso vemos que si el usuario mete un valor incorrecto, toma el error y se vuelve a solicitar el dato que se encuentre en el rango.

```
def pedir_edad(): #Edad
    while True:
        try:
            edad = int(input("Edad: "))
            if edad < 18 or edad > 99: #Edad entre 18-99
                raise EdadInvalidaError(
                    "La edad debe estar entre 18 y 99."
                )
            return edad
        except ValueError:
            print("Error: escribe la edad como un número entero.")
        except EdadInvalidaError as error:
            print("Error:", error)
```

Aquí se aplica el principio “1” **Single Responsibility Principle (Principio de responsabilidad única)**

Se aplica y funciona porque la clase EntradaDatos tiene una responsabilidad única solicitar y validar los datos introducidos por el usuario y no se mezcla con otras clases, como calcular la moneda. Además las excepciones se encuentran separadas, para identificar fácilmente qué tipo de error ocurrió.

2.-Procesamiento por lotes:

En esta parte se realiza el cálculo de prima anual de N asegurados utilizando su Suma asegurada y un factor de edad. El cálculo bien vemos se encuentra separado en la clase calculadoraprima, específicamente dentro de un método.

Primero se obtiene la edad ajustada y posteriormente se obtiene el factor correspondiente utilizando la estrategia de factor de edad y finalmente calculando la prima con la formula anexada al ejercicio.

```
#Factor correspondiente
prima = asegurado.suma_asegurada * factor / 1000
return prima
```

La obtención se encuentra separada mediante nuestra clase abstracta y donde heredan los factores femeninos y masculinos.

Aquí se aplica el principio “2” **O Open/Closed Principle (Principio de abierto/cerrado)**

Se aplica porque nuestra estructura permite agregar(extender) nuevas formas de obtener el factor sin modificar el código ya existente, si quisiéramos agregar otro tipo de factor se puede agregar una nueva clase que heredará del factorstrategy e implementar el método obtener_factor().

3.-Diseño extensible (Open/closed)

En este punto bien como su nombre lo dice buscamos mediante ello que el programa pueda extenderse para agregar nuevos tipos de cálculo o nuevas reglas para determinar el factor de edad sin tener que alterar el código que calcula la prima

esto lo logramos mediante la clase abstracta FactorEdadStrategy que define obtener_factor() y como vemos a partir de ella se crean diferentes estrategias como FactorFemenino con sus propias reglas para calcular el factor correspondiente

```
edad_ajustada = asegurado.calcular_edad_ajustada()
factor = self.estrategia_factor.obtener_factor(edad_ajustada)
```

El código está abierto para extensión pero cerrado para modificación, es decir podemos agregar nuevas estrategias sin modificar nuestra clase ni alterar el código que ya funciona.

Ejemplo si necesito agregar una estrategia distinta únicamente se crearía una nueva clase.

```
class NuevaEstrategia(FactorEdadStrategy):
    def obtener_factor(self, edad):
```

4.-Conversión de moneda desacoplada

Uno de los principios más importantes, en este punto buscamos la conversión de la prima de MXN A USD y no dependa directamente de una tasa de cambio escrita, dentro de la clase que realiza la conversión, además la tasa debe poder sustituirse fácilmente, simulando que proviene de un servicio externo.

Para lograrlo el programa utiliza la clase abstracta Proveedor Tipo Cambio que define el método obtener_tasa() y la clase ConversorMoneda recibe un proveedor de tipo de cambio mediante su constructor.

```
class ConversorMoneda:    #Convierte MXN aUSD
    def __init__(self, proveedor_tipo_cambio):    #Recibe la tasa de cambio
        self.proveedor_tipo_cambio = proveedor_tipo_cambio
```

y se obtiene la tasa mediante

```
tasa = self.proveedor_tipo_cambio.obtener_tasa()    #Se obtiene la tasa
```

además se valida que la tasa de cambio sea válida si es 0 o negativa se genera un error

```
if tasa <= 0:    #Exitar negativos o 0
    raise TasaCambioInvalidaError(
        "La tasa de cambio debe ser mayor que cero."
    )
self.tasa = tasa
```

aquí usamos el principio “5” **D Dependency Inversion Principle (Principio de Inversión de Dependencias)**

en este ejemplo nuestra clase conversor moneda solo se preocupa por convertir la cantidad, no se preocupa ni por cómo ni donde se obtiene la tasa de cambio.

5.-Persistencia

En este último punto se busca que al finalizar el programa, la información de cada asegurado pueda guardarse en un archivo de texto, incluyendo sus características y la prima calculada tanto en MXN como USD.

```
class ExportadorCarnet: #Archivo .txt
    @staticmethod
    def guardar(numero, asegurado, edad_ajustada, prima_mxn, prima_usd):
        #Datos para crear el archivo
        nombre_archivo = f"carnet_{numero}.txt"
```

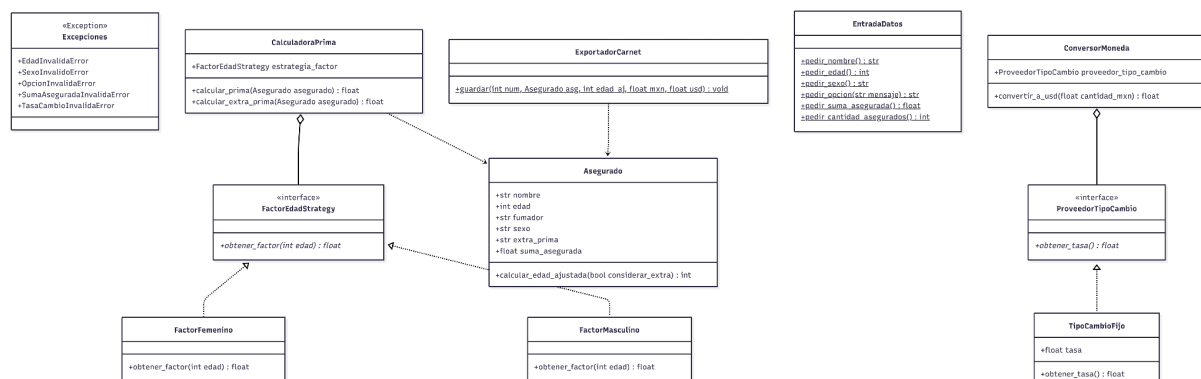
el método crea un archivo de texto y guarda cada dato de cada asegurado.

además utilizamos try/except para manejar posibles errores durante la escritura.

Se aplica el principio “1” **S Single Responsibility Principle (Principio de responsabilidad única)**.

Porque ExportadorCarnet Tiene una responsabilidad específica: guardar la información del asegurado en un archivo.

DIAGRAMA DE CLASES



EJERCICIO 3 DICCIONARIO

1. DESCRIPCIÓN DE LA SOLUCIÓN

El programa recibe un texto y produce un diccionario: la lista alfabética de palabras distintas con su frecuencia. Cumple la restricción del enunciado: no se usa `set`, `sorted()` ni `list.sort()`.

El flujo es: `extraer_palabras` → `TablaHashFrecuencia` (elimina repetidos y cuenta) → `Ordenador` (`MergeSort` / `QuickSort`) → reporte con frecuencias.

Normalización y excepciones. `extraer_palabras` unifica la representación con NFC y aplica `casefold` (sin distinguir mayúsculas), descompone con NFD y descarta las marcas diacríticas (categoría Mn) para quitar acentos, conservando la «ñ». Una palabra es una secuencia de caracteres alfabéticos; números, signos y espacios extra actúan como separadores. Lanza `TextoVacioError` si el texto está vacío o solo tiene espacios, `TextoSinPalabrasError` si no contiene ninguna palabra y `TypeError` si la entrada no es cadena. `main()` las captura con `try/except` y vuelve a pedir el texto.

DIAGRAMA DE CLASES

2. EJEMPLO DE EJECUCIÓN

Entrada :

Adventures in Disneyland. Two blondes were going to Disneyland when they came to a fork in the road. The sign read: "Disneyland LEFT." So they went home.

Se extrajeron 27 palabras, de las cuales 21 son distintas. Salida (lista alfabética sin repetidos), idéntica a la salida esperada del enunciado: a, adventures, blondes, came, disneyland, fork, going, home, in, left, read, road, sign, so, the, they, to, two, went, were, when.

Frecuencia de cada palabra distinta

Palabra	Frecuencia	Palabra	Frecuencia	Palabra	Frecuencia
a	1	home	1	the	2
adventures	1	in	2	they	2
blondes	1	left	1	to	2
came	1	read	1	two	1
dysneyland	3	road	1	went	1
fork	1	sign	1	were	1
going	1	so	1	when	1

Colisiones reales en este ejemplo. La tabla usó capacidad $2 \cdot 27 + 1 = 55$ cubetas y se produjeron 2 colisiones: en la cubeta 32 cayeron «disneyland» y «road», y en la cubeta 51 «two» y «came». Ambas parejas se guardan en la lista de su cubeta y se distinguen comparando la palabra completa, por lo que «disneyland» conserva su frecuencia correcta (3).

3. TABLA HASH PROPIA Y RESOLUCIÓN DE COLISIONES

TablaHashFrecuencia es una lista de listas (buckets). La función hash es $\text{codigo} = (\text{codigo} * 31 + \text{ord}(c)) \% \text{capacidad}$, recorrida carácter por carácter, y devuelve un índice de cubeta. Cada entrada guardada es [palabra, frecuencia].

Colisiones (encadenamiento separado). Si dos palabras distintas producen el mismo índice, ambas se almacenan en la lista de esa cubeta. Al agregar o consultar una palabra solo se recorre su cubeta: si la palabra ya está, se incrementa su frecuencia; si no, se añade [palabra, 1]. Así nunca se compara una palabra contra todas las demás. La capacidad se

fija en $2n+1$, de modo que el factor de carga es ≈ 0.5 y las cubetas se mantienen muy cortas (en el ejemplo, la más larga tuvo 2 elementos).

4. COMPARACIÓN EMPÍRICA DE TIEMPOS

Metodología. Se generaron textos sintéticos de 100, 1,000 y 10,000 palabras (palabras distintas de tres letras, con la primera repetida al final, por lo que hay $n-1$ palabras distintas). Se midió con `time.perf_counter()`: (a) crear la tabla hash e insertar las n palabras, (b) Merge Sort y (c) Quick Sort sobre la misma lista de palabras distintas, tal como las entrega la tabla hash. Cada valor es el promedio de 20 repeticiones (Python 3.12.3); en cada corrida se verificó que ambos algoritmos produjeran el mismo resultado. Los tiempos absolutos dependen de la máquina; lo relevante es cómo crecen.

Palabras de texto	Palabras Distintas	Tabla hash (s)	Merge Sort (s)	Quick Sort (s)
100	99	0.000040	0.000059	0.000021
1,000	999	0.000475	0.000722	0.000315
10,000	9,999	0.005318	0.009244	0.006481

Tabla . Tiempos promedio de ejecución (segundos)

Algoritmo	Teórico 100 - 1,000	Medido 100 - 1,000	Teórico 1,000 - 10,000	Medido 1,000 - 10,000
Tabla hash($O(n)$)	x10.0	x11.7	x10.0	x11.2
Merge Sort($O(n \log n)$)	x15.0	x12.2	13.3	x12.8
Quick Sort($O(n \log n)$ prom.)	x15.0	x14.7	13.3	x20.5

Tabla . Cuanto crece el tiempo al multiplicar por 10 el tamaño de la entrada (teoría vs medición)

5. ANÁLISIS DE COMPLEJIDAD (Big- O)

Sea n el número de palabras del texto y $k \leq n$ el número de palabras distintas. La longitud de una palabra se considera constante (las palabras son cortas), por lo que comparar o calcular el hash de una palabra cuesta $O(1)$.

Algoritmo	Mejor caso	Caso promedio	Caso peor	Memoria extra
Tabla hash (procesar n palabras)	$O(n)$	$O(n)$	$O(n^2)$	$O(k)$
Merge Sort (k palabras)	$O(k \log k)$	$O(k \log k)$	$O(k \log k)$	$O(k)$
Quick Sort (k palabras)	$O(k \log k)$	$O(k \log k)$	$O(k^2)$	$O(\log k)$ prom. / $O(k)$ peor

Tabla . Complejidad temporal y espacial

Tabla hash :Insertar o buscar cuesta $O(1)$ en promedio porque solo se recorre una cubeta; procesar n palabras es $O(n)$. El peor caso ocurre si todas las palabras caen en la misma cubeta: cada inserción recorre una lista de hasta k elementos y el total es $O(n \cdot k) \subseteq O(n^2)$. Con capacidad $2n+1$ el factor de carga es ≈ 0.5 , así que ese peor caso es muy poco probable.

Merge Sort : Divide siempre por la mitad ($\approx \log_2 k$ niveles) y mezclar cuesta $O(k)$ por nivel, de modo que es $O(k \log k)$ en mejor, promedio y peor caso, sin importar el orden de entrada. Requiere $O(k)$ de memoria auxiliar para las listas de mezcla.

Quick Sort : Es $O(k \log k)$ cuando las particiones quedan equilibradas. El peor caso, $O(k^2)$, aparece si el pivote deja partes de tamaño $k-1$ y 0 en cada paso. Elegir el elemento central como pivote evita ese caso con listas ya ordenadas, pero no lo elimina para todas las permutaciones. Ordena sobre una copia (memoria $O(k)$) y la pila de recursión usa $O(\log k)$ en promedio y $O(k)$ en el peor caso.

Costo total del programa : $O(n + k \log k)$: un recorrido lineal para extraer y deduplicar, más el ordenamiento de las k palabras distintas.

Contraste con lo medido.

Tabla hash : el tiempo creció $\approx \times 11-12$ cada vez que la entrada se multiplicó por 10, muy cerca del $\times 10$ lineal esperado. En este experimento la cubeta más larga tuvo un solo elemento (sin colisiones), así que se observó el caso promedio/mejor, no el peor.

Merge Sort : creció $\times 12.2$ y $\times 12.8$, coherente con $n \log n$ ($\times 15.0$ y $\times 13.3$ en teoría; en ambos casos algo menos por costos constantes).

Quick Sort : fue más rápido que Merge Sort en los tres tamaños, pues sus constantes son menores (ordena en el sitio, mientras Merge Sort crea sublistas y listas de mezcla). Sin embargo, de 1,000 a 10,000 palabras creció $\times 20.5$, más que el $\times 13.3$ esperado, aunque muy lejos del $\times 100$ que implicaría un comportamiento cuadrático. Como prueba adicional, al ordenar la misma lista revuelta al azar, Quick Sort pasó de 613 μs a 8,496 μs ($\times 13.9$) y Merge Sort de 1,100 μs a 16,837 μs ($\times 15.3$), cifras acordes con $n \log n$. Esto sugiere que la desviación se debe al orden en que la tabla hash entrega las palabras y a efectos de memoria (caché), y no a un peor caso algorítmico. Con la entrada revuelta ambos algoritmos son más lentos que con el orden de la tabla hash.

Conclusión: las mediciones confirman el orden de crecimiento teórico (lineal para el hash, $\approx n \log n$ para ambos ordenamientos). El Big-O describe cómo crece el tiempo, no cuál algoritmo es más rápido con tamaños concretos: por eso Quick Sort, con el mismo $O(k \log k)$ promedio, resultó más veloz que Merge Sort en la práctica, a cambio de no garantizar el peor caso.

6. APLICACIÓN DE LOS PRINCIPIOS SOLID EN EL CÓDIGO

Se indica la clase o método concreto donde se aplica cada principio, junto con sus limitaciones actuales.

Principio	Donde se aplica	Como se aplica
S Responsabilidad única	extraer_palabras TablaHashFrecuencia MergeSort, QuickSort medir_tabla_hash, medir_ordenamiento	Cada clase o función tiene una sola razón para cambiar. extraer_palabras solo valida, normaliza y separa; TablaHashFrecuencia solo elimina repetidos y cuenta (agregar, palabras_unicas, frecuencia), sin ordenar ni imprimir; cada ordenador solo ordena; las funciones medir_* solo miden tiempo. Cambiar las reglas de limpieza del texto solo toca extraer_palabras. Limitación: main() y comparar_tiempos() mezclan orquestación con impresión.
O Abierto/Cerrado	Clase abstracta Ordenador medir_ordenamiento ciclo sobre la lista ordenadores en main()	El sistema está abierto a extensión y cerrado a modificación: para agregar un tercer algoritmo (p. ej. HeapSort) basta crear una subclase de Ordenador que implemente ordenar() y añadirla a la lista; MergeSort, QuickSort, medir_ordenamiento y el ciclo de main() no se modifican. Limitación: comparar_tiempos() instancia explícitamente los dos algoritmos y su tabla tiene una columna por cada uno.
L Sustitución de Liskov	MergeSort y QuickSort como subtipos de Ordenador	Ambas subclases cumplen el mismo contrato: reciben una lista de cadenas y devuelven una nueva lista ordenada, sin modificar la original (Merge Sort trabaja con sublistas; Quick Sort, sobre palabras.copy()). Por eso son intercambiables donde se espera un Ordenador, y el programa lo comprueba con la verificación de igualdad de resultados que lanza AssertionError.
I Segregación de interfaces	Ordenador.ordenar interfaz pública de TablaHashFrecuencia	Ordenador expone un único método abstracto, ordenar(): quien solo necesita ordenar no depende de métodos de conteo ni de medición. La tabla hash ofrece una interfaz pequeña y separada (agregar, palabras_unicas, frecuencia), y los detalles internos (_calcular_indice, _mezclar, _quick_sort) son privados por convención. Ningún cliente está obligado a depender de métodos que no
D Inversión de dependencias	medir_ordenamiento(or denador, ...) ciclo de main()	Los módulos de alto nivel dependen de la abstracción Ordenador, no de MergeSort ni QuickSort: el objeto se recibe como parámetro (inyección) y las clases concretas solo se instancian en el punto de composición (main /comparar_tiempos). Limitación: TablaHashFrecuencia se usa como clase concreta; si se quisiera intercambiar la estrategia de deduplicación, habría que abstraerla también.

